

MIPS Simulator



CS 3339.254

By: Aidan Liston, Jessica Martinez, Joel Sarmiento

Overview

- Simulated 5-stage pipelined MIPS processor
- Written in C++
- Parses and executes MIPS assembly files, supporting 13 different instructions
- Implements all 5 pipeline stages (IF, ID, EX, MEM, WB) with state registers
- Supports a debug mode that prints pipeline state and control signals each cycle

Pipeline Overview

- **IF/ID**: Holds fetched instruction and current PC
- **ID/EX**: Holds decoded instruction, register values, and immediate
- **EX/MEM**: Holds ALU result and rt value for SW
- **MEM/WB**: Can hold either ALU result or the loaded memory value to be written back to register file

```
// if/id reg: holds fetched instruction and current PC
struct IFID {
    Instructions instr;
    int pc;
    bool nop;
};

// id/ex reg: holds decoded instruction, reg values, and immediate
struct IDEX {
    Instructions instr;
    int pc;
    int rsVal; // value read from rs reg
    int rtVal; // value read from rt reg
    int imm;
    bool nop;
};

// ex/mem reg: holds ALU result and rt value for SW
struct EXMEM {
    Instructions instr;
    int aluResult;
    int rtVal; // rt value forwarded for SW
    bool nop;
};

// mem/wb reg: holds data from memory stage to be written back
struct MEMWB {
    Instructions instr;
    int memData; // data loaded from memory (LW)
    int aluResult; // passed through for non-memory instructions
    bool nop;
};

// global program and pipeline state
extern vector<Instructions> program;
extern int PC;
extern IFID ifid;
extern IDEX idex;
extern EXMEM exmem;
extern MEMWB memwb;

// init all pipeline registers to NOP
void initCPUState();

// advance pipeline by one clock cycle
void stepCycle(bool debug);

// returns true when all stages are empty and PC is past end
bool pipelineEmpty();
```

Memory & Instructions

```
const int NUM_REGS = 32; // 32 general purpose registers
const int MEM_SIZE = 1024; // mem size in words

extern int R[NUM_REGS]; // register file
extern int MEM[MEM_SIZE]; // main memory

// zero out all registers
void initRegisters() {
    for(int i = 0; i < NUM_REGS; i++) {
        R[i] = 0;
    }
}

// zero out all memory
void initMemory() {
    for(int i = 0; i < MEM_SIZE; i++) {
        MEM[i] = 0;
    }
}
```

MEMORY FILE

- 32 general purpose registers
- 1024 words of main memory
- Both zeroed out on startup

```
// MIPS instructions
enum Opcode {
    ADD, ADDI, SUB, MUL, AND_OP, OR_OP, SLL, SRL, LW, SW, BEQ, J, NOP_OP
};

// single decoded MIPS instruction
struct Instructions {
    Opcode op;
    int rs;
    int rt;
    int rd;
    int imm;
    int address;
};

// NOP instruction for filling empty pipeline stages
Instructions makeNOP() {
    Instructions nop;

    nop.op = NOP_OP;
    nop.rs = 0;
    nop.rt = 0;
    nop.rd = 0;
    nop.imm = 0;
    nop.address = 0;

    return nop;
}
```

INSTRUCTIONS FILE

- Each instruction stored as a struct with opcode, register fields, immediate, and address
- makeNOP() used in pipeline on startup, after branch, or when program ends

Initializing CPU

- Reset entire CPU to starting state before execution by filling all stages with NOPs
- Simulation stops when no instructions left to fetch or in pipeline
- StepCycle() executes one complete clock cycle
- All stages read current state and write to “next” copies at the end of each cycle to avoid overwriting data

```
// global pipeline state registers and program
vector<Instructions> program;
int PC = 0;
IFID ifid;
IDEX index;
EXMEM exmem;
MEMWB memwb;

// initialize all pipeline registers to NOP and reset PC
void initCPUState() {
    PC = 0;

    ifid.instr = makeNOP();
    ifid.pc = 0;
    ifid.nop = true;

    index.instr = makeNOP();
    index.pc = 0;
    index.rsVal = 0;
    index.rtVal = 0;
    index.imm = 0;
    index.nop = true;

    exmem.instr = makeNOP();
    exmem.aluResult = 0;
    exmem.rtVal = 0;
    exmem.nop = true;

    memwb.instr = makeNOP();
    memwb.memData = 0;
    memwb.aluResult = 0;
    memwb.nop = true;
}

static bool isNOP(const Instructions& ins) {
    return ins.op == NOP_OP;
}

bool pipelineEmpty() {
    bool allNops = isNOP(ifid.instr) && isNOP(index.instr) && isNOP(exmem.instr) && isNOP(memwb.instr);
    bool pcPastEnd = (PC >= (int)program.size());
    return allNops && pcPastEnd;
}

void stepCycle(bool debug) {
    // copy current pipeline registers into next cycle registers
    IFID next_ifid = ifid;
    IDEX next_index = index;
    EXMEM next_exmem = exmem;
    MEMWB next_memwb = memwb;
}
```

Pipeline Stages: WB & MEM

```
// memory: perform load/store operations
if(!exmem.nop && !isNOP(exmem.instr)) {
    Instructions ins = exmem.instr;
    next_memwb.instr = ins;
    next_memwb.nop = false;
    next_memwb.aluResult = exmem.aluResult;
    next_memwb.memData = 0;

    switch (ins.op) {
        // load word from memory at ALU computed address
        case LW: {
            int address = exmem.aluResult;
            if (address >= 0 && address < MEM_SIZE) {
                next_memwb.memData = MEM[address];
            }
            break;
        }
        // store word to memory at ALU-computed address
        case SW: {
            int address = exmem.aluResult;
            if (address >= 0 && address < MEM_SIZE) {
                MEM[address] = exmem.rtVal;
            }
            break;
        }
        default:
            break;
    }
} else {
    next_memwb.instr = makeNOP();
    next_memwb.nop = true;
}
```

MEM STAGE

- Where load and store operations are done
- Addresses are bounds-checked

```
// writeback: write ALU result or memory data back to register file
if(!memwb.nop && !isNOP(memwb.instr)) {
    Instructions ins = memwb.instr;
    switch (ins.op) {
        // r type instructions write to rd
        case ADD: case SUB: case MUL: case AND_OP: case OR_OP: case SLL: case SRL:
            if (ins.rd != 0) {
                R[ins.rd] = memwb.aluResult;
            }
            break;
        // ADDI writes to rt
        case ADDI:
            if (ins.rt != 0) {
                R[ins.rt] = memwb.aluResult;
            }
            break;
        // LW writes loaded memory data to rt
        case LW:
            if (ins.rt != 0) {
                R[ins.rt] = memwb.memData;
            }
            break;
        default:
            break;
    }
}
```

WRITEBACK STAGE

- R-type writes ALU result to rd, I-type writes to rt
- LW writes loaded memory data instead of ALU result

Execution

- Reads rsVal, rtVal, and imm from ID/EX
- Performs ALU operation based on opcode
- LW/SW compute memory address as base register+offset

```
// execution: ALU operations
if(!idex.nop && !isNOP(idex.instr)) {
    Instructions ins = idex.instr;
    next_exmem.instr = ins;
    next_exmem.nop = false;
    next_exmem.rtVal = idex.rtVal;

    int a = idex.rsVal;
    int b = idex.rtVal;
    int imm = idex.imm;
    int result = 0;

    // ALU operation
    switch(ins.op) {
        case ADD:
            result = a + b;
            break;
        case ADDI:
            result = a + imm;
            break;
        case SUB:
            result = a - b;
            break;
        case MUL:
            result = a * b;
            break;
        case AND_OP:
            result = a & b;
            break;
        case OR_OP:
            result = a | b;
            break;
        case SLL:
            result = b << imm;
            break;
        case SRL:
            result = (unsigned int)b >> imm;
            break;
        case LW:
        case SW:
            result = a + imm;
            break;
        default:
            break;
    }

    next_exmem.aluResult = result;
} else {
    next_exmem.instr = makeNOP();
    next_exmem.nop = true;
}
```


Instruction Fetch & Decode

```
// instruction decode: read register values and handle branches/jumps early
if(!ifid.nop && !isNOP(ifid.instr)) {
    Instructions ins = ifid.instr;

    next_idx.instr = ins;
    next_idx.nop = false;
    next_idx.pc = ifid.pc;
    next_idx.imm = ins.imm;

    // defaults to 0 if value read is out of bounds
    if (ins.rs >= 0 && ins.rs < NUM_REGS) {
        next_idx.rsVal = R[ins.rs];
    } else {
        next_idx.rsVal = 0;
    }

    if (ins.rt >= 0 && ins.rt < NUM_REGS) {
        next_idx.rtVal = R[ins.rt];
    } else {
        next_idx.rtVal = 0;
    }
}
```

INSTRUCTION DECODE

- Reads rs and rt register values into rsVal and rtVal
- Passes instruction, immediate, and register values to IDEX

```
// BEQ: resolve branch in id stage and flush if taken
if(ins.op == BEQ) {
    int a = next_idx.rsVal;
    int b = next_idx.rtVal;
    if(a == b) {
        PC = ins.address;
        // flush instructions that enter pipeline after branch
        next_ifid.instr = makeNOP();
        next_ifid.nop = true;
        next_idx.instr = makeNOP();
        next_idx.nop = true;
        next_idx.rsVal = 0;
        next_idx.rtVal = 0;
        next_idx.imm = 0;
        next_exmem.instr = makeNOP();
        next_exmem.nop = true;
    }
}

// J: unconditional jump so always flush
} else if (ins.op == J) {
    PC = ins.address;
    next_ifid.instr = makeNOP();
    next_ifid.nop = true;
    next_idx.instr = makeNOP();
    next_idx.nop = true;
    next_idx.rsVal = 0;
    next_idx.rtVal = 0;
    next_idx.imm = 0;
    next_exmem.instr = makeNOP();
    next_exmem.nop = true;
}
} else {
    next_idx.instr = makeNOP();
    next_idx.nop = true;
    next_idx.rsVal = 0;
    next_idx.rtVal = 0;
    next_idx.imm = 0;
}
```

BRANCH & JUMP

- BEQ compares rsVal, rtVal, jumps and flushes pipeline if equal
- J always jumps and flushes, NOPs injected into IFID, IDEX, EXMEM

```
// instruction fetch: fetch next instruction from program using PC
if(PC < (int)program.size()) {
    next_ifid.instr = program[PC];
    next_ifid.pc = PC;
    next_ifid.nop = false;
    PC++;
} else {
    next_ifid.instr = makeNOP();
    next_ifid.pc = PC;
    next_ifid.nop = true;
}

// next pipeline state
ifid = next_ifid;
idx = next_idx;
exmem = next_exmem;
memwb = next_memwb;
```

INSTRUCTION FETCH

- Fetches next instruction from program using PC
- Increments PC each cycle, inserts NOP when past end of program

Parser Overview

- Reads the input from the provided .asm file
- Removes comments and whitespaces
- Handles label to address mapping
- Makes a vector on instructions for the CPU
- Converts assembly text to structured instruction fields
- Two pass throughs for parsing

```
bool loadProgram(const string &filename, vector<Instructions> &outProgram) {  
    ifstream in(filename);  
    if (!in) {  
        cerr << "error: cannot open input file '" << filename << "'" << endl;  
        return false;  
    }  
  
    outProgram.clear();  
  
    // holds instruction text after labels removed  
    vector<string> instructionLines;  
  
    // maps label name to instruction index (PC)  
    unordered_map<string, int> labels;  
  
    string line;  
    // counts how many instructions we've seen  
    int instructionIndex = 0;  
  
    // original file line number  
    int lineNumber = 0;
```

Pass 1: Label Collection

- Remove comments
- Detect Labels
- Map labels to their instruction index
- Store instruction lines without labels

```
// pass 1 creates a map from labels to PC index
while (getline(in, line)) {
    lineNumber++;

    // remove comments (anything after # or ;)
    size_t pos = line.find('#');
    if (pos != string::npos) {
        line = line.substr(0, pos);
    }

    pos = line.find(';');
    if (pos != string::npos) {
        line = line.substr(0, pos);
    }

    // trim whitespace and skip empty lines
    line = trim(line);
    if (line.empty()) {
        continue;
    }

    // find labels
    pos = line.find(':');
    if (pos != string::npos) {
        string label = trim(line.substr(0, pos));
        if (!label.empty()) {
            // check for duplicate labels
            if (labels.count(label)) {
                cerr << "Error: duplicate label '" << label << "' on line " << lineNumber << endl;
                return false;
            }

            // map label to the current instruction index
            labels[label] = instructionIndex;
        }

        // remove label and look for instruction
        line = trim(line.substr(pos + 1));
    }

    // if there's still text left then it's an instruction line
    if (!line.empty()) {
        instructionLines.push_back(line);
        instructionIndex++;
    }
}
```

Pass 2: Instruction Decode

- Uppercase opcode lowercase registers
- Parse R-type(3 regs), I-type(2 reg, 1 Imm), J-type(1 target adrs) formats
- Converts register strings to register numbers using helper func `parseRegister` that maps registers to numbers
- Maps BEQ/J labels to numeric PC targets
- Fill out the instructions struct (op, rs, rt etc.)

```
pass 2 parses opcodes and uses the map
for (int i = 0; i < (int)instructionLines.size(); i++) {
    string s = instructionLines[i];

    // replace commas with spaces to make splitting easier
    replace(s.begin(), s.end(), ',', ' ');

    stringstream ss(s);

    // read opcode
    string opStr;
    ss >> opStr;

    if (opStr.empty()) {
        cerr << "error: missing opcode at instruction " << i << endl;
        return false;
    }

    string opUpper = toUpper(opStr);

    // initialize new instruction with default values
    Instructions ins{};
    ins.op = NOP_OP;
    ins.rs = 0;
    ins.rt = 0;
    ins.rd = 0;
    ins.imm = 0;
    ins.address = 0;

    // r type
    if (opUpper == "ADD" || opUpper == "SUB" || opUpper == "MUL" || opUpper == "AND" || opUpper == "OR") {
        string rdTok, rsTok, rtTok;
        ss >> rdTok >> rsTok >> rtTok;

        // convert register tokens to register numbers
        int rd = parseRegister(rdTok);
        int rs = parseRegister(rsTok);
        int rt = parseRegister(rtTok);

        if (rd < 0 || rs < 0 || rt < 0) {
            cerr << "error: bad register in " << opUpper << " at instruction " << i << endl;
            return false;
        }

        // map opcode string to enum Opcode
        if (opUpper == "ADD") {
            ins.op = ADD;
        }
        else if (opUpper == "SUB") {
            ins.op = SUB;
        }
        else if (opUpper == "MUL") {
            ins.op = MUL;
        }
        else if (opUpper == "AND") {
            ins.op = AND_OP;
        }
        else if (opUpper == "OR") {
            ins.op = OR_OP;
        }

        // save r type register fields
        ins.rd = rd;
        ins.rs = rs;
        ins.rt = rt;
    }
}
```

Example: Before & After Parsing

- Pass 1: Label Collection (loop ends)
- Pass 2: Instruction Decoding

Line	Text	Instruction Index	Label Table Update
1	start:		start -> 0
2	addi	0	
3	loop:		loop -> 1
4	addi	1	
5	beq	2	
6	j loop	3	
7	end:		end -> 4
8	nop	4	

PC Parsed Fields

```

0      opcode=ADDI, rs=0, rt=8, imm=5
1      opcode=ADDI, rs=9, rt=9, imm=1
2      opcode=BEQ, rs=9, rt=8, address=4
3      opcode=J, address=1
4      nop=true
  
```

start:

```
addi $t0, $zero, 5
```

loop:

```
addi $t1, $t1, 1
```

```
beq $t1, $t0, end
```

```
j loop
```

end:

```
nop
```

```

// named registers
static const unordered_map<string, int> regMap = {
    {"zero", 0}, {"at", 1},
    {"v0", 2}, {"v1", 3},
    {"a0", 4}, {"a1", 5}, {"a2", 6}, {"a3", 7},
    {"t0", 8}, {"t1", 9}, {"t2", 10}, {"t3", 11},
    {"t4", 12}, {"t5", 13}, {"t6", 14}, {"t7", 15},
    {"s0", 16}, {"s1", 17}, {"s2", 18}, {"s3", 19},
    {"s4", 20}, {"s5", 21}, {"s6", 22}, {"s7", 23},
    {"t8", 24}, {"t9", 25},
    {"k0", 26}, {"k1", 27},
    {"gp", 28},
    {"sp", 29},
    {"fp", 30}, {"s8", 30},
    {"ra", 31}
};
  
```

Conclusion

- Built a MIPS pipeline simulator implementing IF,ID,EX,MEM,WB
- Implemented support for all of the required instructions
- We will now show a demo of the program execution using a test assembly file